ANÁLISE DE DESEMPENHO ENTRE PROGRAMAÇÃO REATIVA COM SPRING WEBFLUX/R2DBC E PROGRAMAÇÃO TRADICIONAL COM SPRING MVC/JDBC EM APIS JAVA SOB ALTA CONCORRÊNCIA

Vitor Martins Melo
Athos Lial Passos
Jonatan Mariano da Silva

RESUMO

Este estudo compara APIs Java implementadas com Spring MVC/JDBC e Spring WebFlux/R2DBC em operações SQL e pedidos com comunicação entre serviços. A bancada utiliza containers com cotas de recursos, geração de carga aberta e validação das respostas. Foram preservadas 282 rodadas principais e 20 testes curtos, distinguindo versões, sessões e intervenções diagnósticas. A matriz completa revisada de 31 de outubro de 2026 contém 72 rodadas, taxas de 200, 500 e 1.000 requisições por segundo e três repetições por condição. Suas 3.672.060 requisições de medição foram corretas, sem iterações descartadas. Nas operações SQL, as medianas dos p95 do MVC foram de 0,387 a 0,945 ms inferiores às do WebFlux. Em pedidos a 1.000 req/s, os p95 medianos foram 6,333 ms com RestTemplate, 6,171 ms com OpenFeign e 10,414 ms com WebClient. Sessões anteriores apresentaram degradação reativa, associada a restrição de CPU no diagnóstico; uma intervenção de cota acompanhou melhora, sem isolar todas as causas. A auditoria também identificou diferenças no transporte Feign, falhas de inicialização de conversores e mudança da JVM entre reconstruções. Os resultados caracterizam as implementações e demonstram a importância de preservar proveniência, entrega de carga e variabilidade, sem estabelecer superioridade universal de um paradigma.

Palavras-chave: Spring MVC; Spring WebFlux; JDBC; R2DBC; avaliação de desempenho.

ABSTRACT

This study compares Java APIs implemented with Spring MVC/JDBC and Spring WebFlux/R2DBC for SQL operations and an order workflow involving inter-service communication. The testbed combines container resource quotas, open-model load generation, and response validation. It preserves 282 main runs and 20 smoke tests, separating implementation versions, sessions, and diagnostic interventions. The revised complete matrix collected on October 31, 2026 comprises 72 runs, arrival rates of 200, 500, and 1,000 requests per second, and three repetitions per condition. All 3,672,060 measurement requests were valid, with no dropped iterations. MVC median run-level p95 latencies were 0.387–0.945 ms lower for SQL operations. At 1,000 orders per second, median p95 values were 6.333 ms for RestTemplate, 6.171 ms for OpenFeign, and 10.414 ms for WebClient. Earlier sessions exhibited reactive-stack degradation associated with CPU throttling in the diagnostic session; increasing the quota accompanied improvement without isolating every cause. The audit also identified Feign transport differences, converter-initialization failures, and JVM changes between image rebuilds. The findings characterize the tested implementations and emphasize provenance, delivered load, and variability rather than universal superiority of a programming paradigm.

Keywords: Spring MVC; Spring WebFlux; JDBC; R2DBC; performance evaluation.

1 INTRODUÇÃO

APIs que consultam bancos de dados e outros serviços alternam processamento com espera por entrada e saída. A organização dessa espera influencia a ocupação de threads e pode afetar o desempenho sob concorrência. No ecossistema Java, Spring MVC/JDBC e Spring WebFlux/R2DBC representam combinações distintas de servidor HTTP, modelo de execução e acesso relacional. Compará-las exige controlar a carga e os recursos, sem pressupor que operações não bloqueantes produzam sempre respostas mais rápidas.

O problema de pesquisa é: como essas duas pilhas se comportam em latência, vazão correta e atendimento da carga quando os recursos da aplicação são limitados e a taxa de chegada aumenta? O objetivo é comparar implementações funcionalmente equivalentes em leitura e atualização SQL e, como extensão, em um fluxo de pedidos com consulta HTTP a outro serviço e persistência. Nesse fluxo, distinguem-se RestTemplate e OpenFeign no MVC e WebClient no WebFlux.

A contribuição consiste em uma bancada reproduzível com resultados por rodada e uma análise que distingue integridade numérica, entrega da demanda, variabilidade temporal e mudanças de implementação. A coleta completa mais recente é apresentada junto aos resultados anteriores e às validações posteriores, evitando selecionar apenas a sessão mais favorável. A investigação mostra por que uma diferença observada entre frameworks pode envolver configuração, dependências e estado do ambiente.

2 REFERENCIAL TEÓRICO

2.1 Concorrência e acesso a dados

Spring MVC é construído sobre a API Servlet (Spring, s.d.-b). Na implementação estudada, cada fluxo utiliza threads de plataforma e chamadas JDBC bloqueantes. JDBC fornece interfaces Java para conexões, comandos SQL e leitura de resultados (Oracle, s.d.). A thread que executa uma operação bloqueante aguarda sua conclusão antes de continuar. Concorrência não significa necessariamente paralelismo: várias requisições podem estar em andamento enquanto poucas executam instruções simultaneamente.

WebFlux organiza operações não bloqueantes com um conjunto reduzido de threads, enquanto R2DBC define interfaces de acesso relacional compatíveis com fluxos reativos (Spring, s.d.-a; Hale et al., 2022). Reactive Streams fornece um contrato de demanda entre produtor e consumidor, denominado backpressure; esse contrato não constitui, isoladamente,

um limite global de admissão HTTP ou proteção contra toda sobrecarga (Reactive Streams, s.d.).

A comparação avalia pilhas completas, incluindo servidores, drivers e pools. R2DBC não reduz automaticamente o custo do SQL no PostgreSQL. Da mesma forma, um conjunto menor de threads não assegura menor latência por requisição. A documentação do Spring relaciona benefícios reativos a concorrência e esperas externas, sem prometer ganho automático de velocidade (Spring, s.d.-a).

2.2 Comunicação entre serviços

OpenFeign oferece interfaces declarativas para chamadas HTTP; o modo utilizado neste estudo é bloqueante, assim como RestTemplate (Spring, s.d.-c). WebClient permite integrar chamadas ao fluxo reativo. O trabalho de Mochniej e Badurowicz (2023), sobre microserviços Java imperativos e reativos, relaciona desempenho a atrasos de comunicação e à complexidade das operações. Este estudo não reproduz integralmente aquele desenho: utiliza rede local, consultas curtas e recursos delimitados por container.

Separar serviços acrescenta dependências de rede e de persistência, mas não elimina interferência quando eles compartilham uma máquina. Em pedidos, o resultado da consulta de estoque é necessário à gravação subsequente. Não existem chamadas remotas independentes executadas em paralelo nesse fluxo. Assim, a composição reativa muda a organização da espera, mas preserva dependências sequenciais e trabalho de processamento.

2.3 Métricas e limites de recursos

O p95 descreve a região abaixo da qual ficam aproximadamente 95% das durações observadas; o p99 enfatiza uma cauda mais lenta. Dean e Barroso (2013) discutem a relevância da cauda em serviços compostos. Neste estudo, calcula-se cada percentil por rodada e depois a mediana entre repetições. Essa mediana não é o percentil de todas as requisições agrupadas.

Executores de taxa de chegada do k6 tentam iniciar operações independentemente das conclusões anteriores. Quando não existem usuários virtuais disponíveis, iterações programadas podem não começar, sendo registradas em dropped_iterations (Grafana Labs, s.d.-a, s.d.-b). Essas operações não têm latência HTTP. Por isso, comparar apenas p95 pode favorecer uma configuração que deixou de executar parte da carga.

No Linux, a cota de CPU de um cgroup representa orçamento de execução por período, não reserva exclusiva de núcleos. O contador nr_throttled registra períodos com restrição após

esgotamento do orçamento (Linux Kernel, 2023). Uma média de utilização abaixo da cota não exclui restrições breves; CPU, threads, conexões e capacidade do gerador são limites diferentes.

3 METODOLOGIA

3.1 Cenários e ambiente

Foi conduzido um experimento quantitativo e exploratório com Java 21, Spring Boot 4.0.3 e Spring Cloud 2025.1.1 na integração OpenFeign. A leitura consulta um produto por chave primária em uma tabela de 10.000 registros. A escrita usa UPDATE com RETURNING e valor variável, produzindo alterações reais. Nesses dois cenários não há cliente HTTP externo. O fluxo de pedidos consulta estoque, valida dados, calcula o valor total e insere um pedido em PostgreSQL próprio. Não implementa pagamento nem baixa distribuída de estoque.

O serviço de estoque, comum às variantes, utiliza banco separado. O controle inventory consulta diretamente esse serviço; ele não executa todo o trabalho de um pedido. Os cenários analisados não introduzem atraso artificial, apesar de existir no laboratório outro endpoint destinado a esse fim. A matriz completa possui oito braços: dois de leitura, dois de escrita, três de pedidos e um controle. O cruzamento com três taxas e três repetições produz 72 rodadas.

O ambiente utiliza Windows build 26200, Docker Desktop 4.86.0, Engine 29.7.2 e kernel Linux 6.6.87.2-microsoft-standard-WSL2. O Docker registrou 14 processadores lógicos e aproximadamente 31,09 GiB disponíveis. O inventário físico de 29/10 identificou Intel Core Ultra 7 255U, 12 núcleos, 14 processadores lógicos e aproximadamente 63,48 GiB visíveis ao Windows. As cotas da Tabela 1, e não toda a memória do host, delimitam os containers. Aplicações, bancos e gerador continuam disputando o mesmo host.

Tabela 1 – Recursos por componente

Componente	Cota e memória	Configuração
API avaliada	2 CPUs; 1 GiB	Heap 512 MiB; G1; ActiveProcessorCount=2; pool SQL 20
PostgreSQL da API	2 CPUs; 768 MiB	PostgreSQL 17.4; 10.000 produtos
Serviço de estoque	1 CPU; 640 MiB	Heap 256 MiB; G1; ActiveProcessorCount=1
PostgreSQL do estoque	1 CPU; 512 MiB	Banco separado
Gerador k6	2 CPUs; 1 GiB	500 VUs; 60 s de aquecimento + 90 s de medição

Fonte: elaboração dos autores a partir dos registros do experimento.

No diagnóstico histórico, somente as condições identificadas alteram CPU da API ou VUs. O pool HTTP de 100 conexões aplica-se a RestTemplate e WebClient e ao Feign revisado; não ao Feign histórico.

MVC usa até 200 threads de plataforma, com virtual threads desativadas. As APIs usam pool SQL de 20 conexões, heap inicial e máximo de 512 MiB, G1 e ActiveProcessorCount=2. O estoque usa heap de 256 MiB e ActiveProcessorCount=1. RestTemplate utiliza Apache HttpClient 5; WebClient usa Reactor Netty. Na versão revisada, o Feign é ligado explicitamente ao Apache HttpClient 5. Os pools HTTP desses clientes são configurados com 100 conexões. Limites por fase e políticas internas de fila diferem entre implementações; eles não equivalem a um prazo global idêntico.

3.2 Protocolo de coleta e análise

Cada rodada reinicia a aplicação avaliada, prepara os dados, executa 60 s de aquecimento e depois 90 s de medição. O aquecimento usa rampa em 75% da janela e platô nos 25% finais; suas falhas são contadas separadamente. Na escrita SQL, os dados são novamente preparados após aquecer, reduzindo diferenças de tuplas mortas no início da medição. Nas matrizes completas, as condições são embaralhadas com semente 20260928. Há três repetições por combinação de cenário, variante e taxa; cada uma constitui uma unidade experimental.

O gerador k6 v1.0.0-labclock1 usa o protocolo 4 e timing_revision=monotonic-anchor-v1. O patch local de relógio, os scripts e os hashes são preservados. A latência principal corresponde a envio, espera e recebimento HTTP; bloqueio e estabelecimento da conexão ficam fora dessa métrica e são registrados separadamente. Vazão correta divide respostas aprovadas pelo validador pelo tempo efetivamente registrado, incluindo drenagem. Não se presume igualdade exata entre contadores programados, observados e descartados nas fronteiras do agendamento.

A verificação confere contagens de requisições, amostras de duração nativa e respostas corretas, além de recalcular p50, p95 e p99 a partir das séries brutas. Nenhuma amostra foi excluída nos lotes incorporados. A triagem temporal compara três janelas e sinaliza amplitude de p95 superior ao maior entre 5 ms e 20% da mediana, ou amplitude de contagens superior a 10% da mediana. Essa heurística não certifica estacionariedade e os alertas não removem rodadas. O indicador legado de entrega no platô de aquecimento pode incluir conclusões iniciadas na rampa ou na drenagem; por isso, não é usado como prova isolada de estabilização.

3.3 Versões e desenho das sessões

Tabela 2 – Coletas e versões preservadas

Coleta	n[1]	JVM Temurin	Feign e plano de energia
15 e 20/10	102	21.0.12.1+1	Transporte padrão; High performance
29/10	21	21.0.12.1+1	Transporte padrão; Razer Cortex; intervenções
30/10 completa	72	21.0.13+9	HC5 explícito; Razer Cortex; conversores sob demanda
30/10 validação	3	21.0.13+9	HC5; conversores antecipados; Razer Cortex
31/10 validação	3	21.0.13+9	HC5; conversores antecipados; Balanced
31/10 completa	72	21.0.13+9	HC5; conversores antecipados; High performance
01/11 validação	9	21.0.12.1+1	HC5; conversores antecipados; High performance

Fonte: elaboração dos autores a partir dos registros do experimento.

[1] Rodadas principais. Outros 20 testes curtos são preservados, mas excluídos da análise de desempenho. Em 01/11 as imagens base foram fixadas por digest. As sessões não são replicações de uma única configuração.

As 282 rodadas principais abrangem comparação e diagnóstico; não são 282 repetições da mesma condição. As 20 rodadas curtas verificam funcionamento e permanecem fora das estimativas de desempenho. O conjunto principal soma 16.017.093 requisições observadas, com duas respostas inválidas nas sessões históricas. Esses totais servem ao inventário, não a um teste estatístico agregado.

A auditoria encontrou um desvio entre configuração pretendida e transporte efetivo: até 29/10, o bean CloseableHttpClient já existente fazia a autoconfiguração HC5 do OpenFeign recuar. Sem um feign.Client explícito, o cliente utilizava Client.Default, baseado em HttpURLConnection. Portanto, não se atribui retrospectivamente a esse braço o pool HC5 de 100 conexões. O código revisado declara ApacheHttp5Client explicitamente, e testes de contexto verificam tanto o estado anterior quanto o atual.

No lote completo de 30/10, RemoteErrors passou a registrar causas e o transporte já estava corrigido. Após observar falhas de decodificação no aquecimento, adicionou-se FeignDecoderWarmup para inicializar os conversores antes das requisições concorrentes. A matriz completa de 31/10 combina essas correções e plano High performance. Ela é a referência da implementação revisada, enquanto os resultados anteriores e os retestes permanecem discriminados nas tabelas e no suplemento.

Os registros jvm_info mostram Temurin 21.0.13+9 já em 30/10 e em 31/10, em contraste com 21.0.12.1+1 nas sessões históricas. Imagens base referidas por tags móveis permitiram a atualização durante a reconstrução, apesar de Dockerfiles inalterados. Em 01/11, fixaram-se digests e repetiu-se apenas a matriz de pedidos Feign na JVM histórica. Essa

validação não constitui uma nova comparação completa entre frameworks nem isola o efeito da JVM.

3.4 Diagnóstico histórico

Em 29/10 foram mantidas as imagens de 20/10 e realizadas 21 rodadas: nove de referência a 1.000 pedidos/s, três WebFlux a 500/s, três WebFlux com 1.000 VUs, três WebFlux com cota de 4 CPUs, dois perfis JFR e uma volta final do WebFlux à configuração inicial. A alteração de CPU preservou heap, pools e ActiveProcessorCount=2. As intervenções foram sequenciais; somente as variantes de referência tiveram ordem sorteada dentro de cada repetição. O gerador de 15/10 estava indisponível, deixando três rodadas comparativas previstas sem execução.

O percentual de períodos com restrição foi calculado por 100 × Δnr_throttled / Δnr_periods, usando cpu.stat antes e depois da medição. A janela inclui pequeno tempo operacional do script; não é percentual de tempo de parede perdido. Perfis JFR incluem inicialização e aquecimento e não foram usados para atribuir custo a métodos. O plano de energia nessa sessão era Razer Cortex, diferentemente das coletas históricas em High performance.

4 RESULTADOS E DISCUSSÃO

4.1 Qualidade das coletas revisadas

Tabela 3 – Integridade e entrega nas novas coletas

Coleta	n	Requisições	Erros[1]	Descartes[1]	Erros[2]
30/10 completa	72	3.665.223	0	6833	186
30/10 validação	3	269.336	0	668	0
31/10 validação	3	269.724	0	278	0
31/10 completa	72	3.672.060	0	0	0
01/11 validação	9	456.522	0	2485	0

Fonte: elaboração dos autores a partir dos registros do experimento.

[1] Medição. [2] Aquecimento. As duas fases são separadas. Todas as novas rodadas tiveram zero amostras excluídas; testes curtos ficam fora desta tabela.

A matriz completa de 31/10 registrou 3.672.060 respostas corretas frente a 3.672.000 requisições programadas, com zero erros, zero descartes e zero amostras excluídas. Seu aquecimento registrou 1.620.896 respostas corretas, sem erros ou descartes. A triagem temporal

sinalizou seis das 72 medições; portanto, integridade e atendimento da carga não significam ausência de variação durante todas as rodadas.

Em 30/10, a matriz completa também teve zero erros de medição, mas 6.833 iterações não iniciadas, todas em pedidos WebFlux a 1.000/s. Os retestes restritos ao Feign tiveram zero respostas inválidas de medição, porém registraram descartes. Essa distinção impede apresentar a correção de uma falha de resposta como solução automática de todos os limites de capacidade.

4.2 Operações SQL na implementação revisada

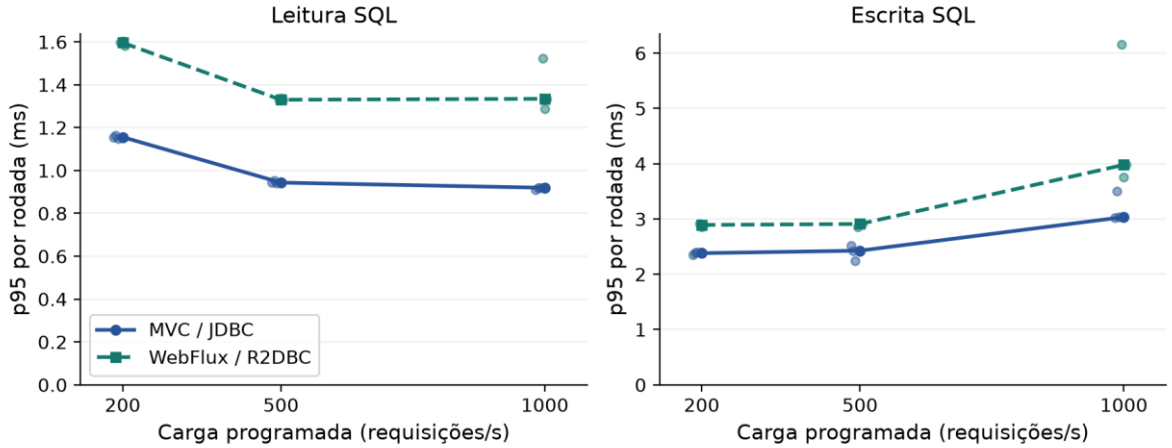
Tabela 4 – Medianas dos p95 SQL em 31 de outubro

Cenário	req/s	MVC (ms)	WebFlux (ms)	Diferença (ms)
Leitura	200	1,156	1,596	0,440
Leitura	500	0,943	1,330	0,387
Leitura	1000	0,919	1,334	0,415
Escrita	200	2,382	2,892	0,511
Escrita	500	2,427	2,910	0,482
Escrita	1000	3,035	3,979	0,945

Fonte: elaboração dos autores a partir dos registros do experimento.

Três repetições por condição; diferença = WebFlux − MVC. Medianas de p95 não equivalem ao p95 de todas as requisições agrupadas.

Figura 1 – Operações SQL na coleta revisada de 31 de outubro



Fonte: elaboração dos autores. Linhas e marcadores: medianas; pontos menores: três repetições, deslocadas horizontalmente para leitura. Cargas em escala numérica. Segmentos guiam a leitura; cargas intermediárias não foram medidas.

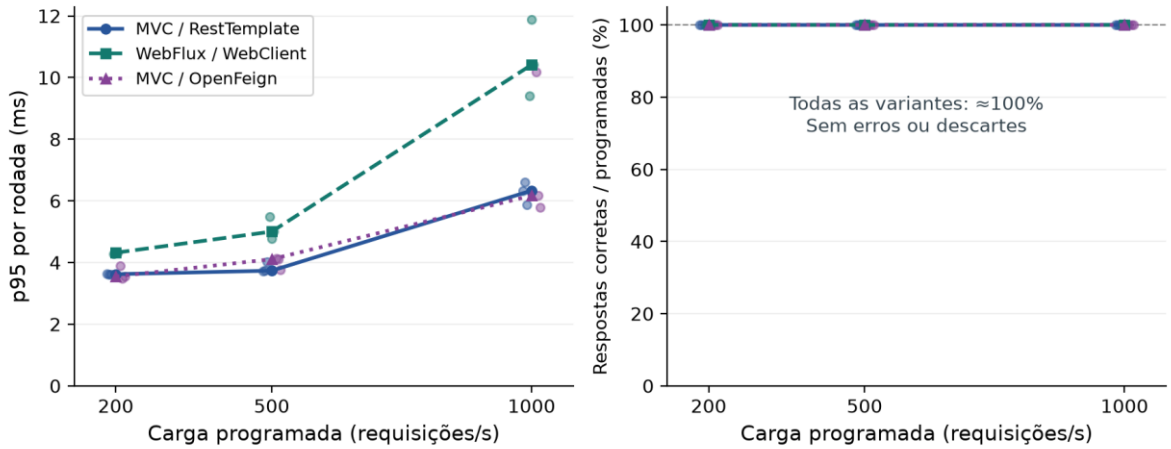
Nas seis condições SQL de 31/10, MVC/JDBC apresentou menores medianas de p95, com diferenças de 0,387 a 0,945 ms. A leitura a 1.000 req/s teve medianas de 0,919 ms no MVC e 1,334 ms no WebFlux; a escrita, 3,035 e 3,979 ms. Ambas as pilhas executaram as cargas sem

erros ou descartes. O tamanho absoluto das diferenças deve ser considerado antes de atribuir relevância prática a uma razão percentual entre valores pequenos.

No lote de 15/10, a mesma direção das medianas havia sido observada, com diferenças de 0,237 a 0,669 ms. Esse histórico reforça a descrição do recorte, mas não aumenta artificialmente n pela mistura de sessões com JVM e ambiente diferentes. As observações individuais e as amplitudes não demonstram significância estatística ou equivalência. Vazões próximas ao alvo imposto não representam capacidade máxima.

4.3 Pedidos e variação entre sessões

Figura 2 – Pedidos na coleta revisada de 31 de outubro



Fonte: elaboração dos autores. Linhas e marcadores: medianas; pontos menores: três repetições. Segmentos guiam a leitura, sem medições intermediárias. Entrega próxima a 100% faz as linhas se sobreporem no painel direito; não significa capacidade máxima.

A 1.000 pedidos/s em 31/10, as medianas dos p95 foram 6,333 ms com RestTemplate, 6,171 ms com OpenFeign e 10,414 ms com WebClient. As três variantes acompanharam a taxa programada sem descartes ou respostas inválidas. A 500/s, as medianas foram 3,735, 4,108 e 5,002 ms, respectivamente. Assim, a posição relativa entre os dois clientes MVC também depende da condição; uma pequena diferença não estabelece superioridade geral.

Figura 3 – Pedidos a 1.000 req/s entre sessões e versões

Latência p95 (ms) — menor é melhor

	15/10	20/10	29/10	30/10	31/10
MVC / RestTemplate	**8,0** 6,9–11,1	**15,8** 15,5–19,6	**19,4** 10,3–24,1	**10,7** 10,5–12,6	**6,3** 5,9–6,6
WebFlux / WebClient	**19,2** 13,5–20,2	**603,3** 579,2–710,9	**627,8** 429,9–737,1	**580,6** 518,5–587,0	**10,4** 9,4–11,9
MVC / OpenFeign	**8,2** 7,9–8,9	**16,6** 16,2–140,2	**11,0** 9,8–14,0	**11,8** 9,7–12,6	**6,2** 5,8–10,2

Vazão correta (req/s) — maior é melhor

	15/10	20/10	29/10	30/10	31/10
MVC / RestTemplate	**1000,0** 999,8–1000,0	**1000,0** 1000,0–1000,0	**1000,0** 1000,0–1000,0	**1000,0** 999,9–1000,0	**1000,0** 1000,0–1000,0
WebFlux / WebClient	**1000,0** 999,9–1000,0	**960,2** 903,1–970,8	**957,4** 901,3–997,7	**969,8** 968,0–986,2	**1000,0** 1000,0–1000,0
MVC / OpenFeign	**1000,0** 1000,0–1000,0	**1000,0** 1000,0–1000,0	**1000,0** 1000,0–1000,0	**1000,0** 1000,0–1000,0	**1000,0** 1000,0–1000,0

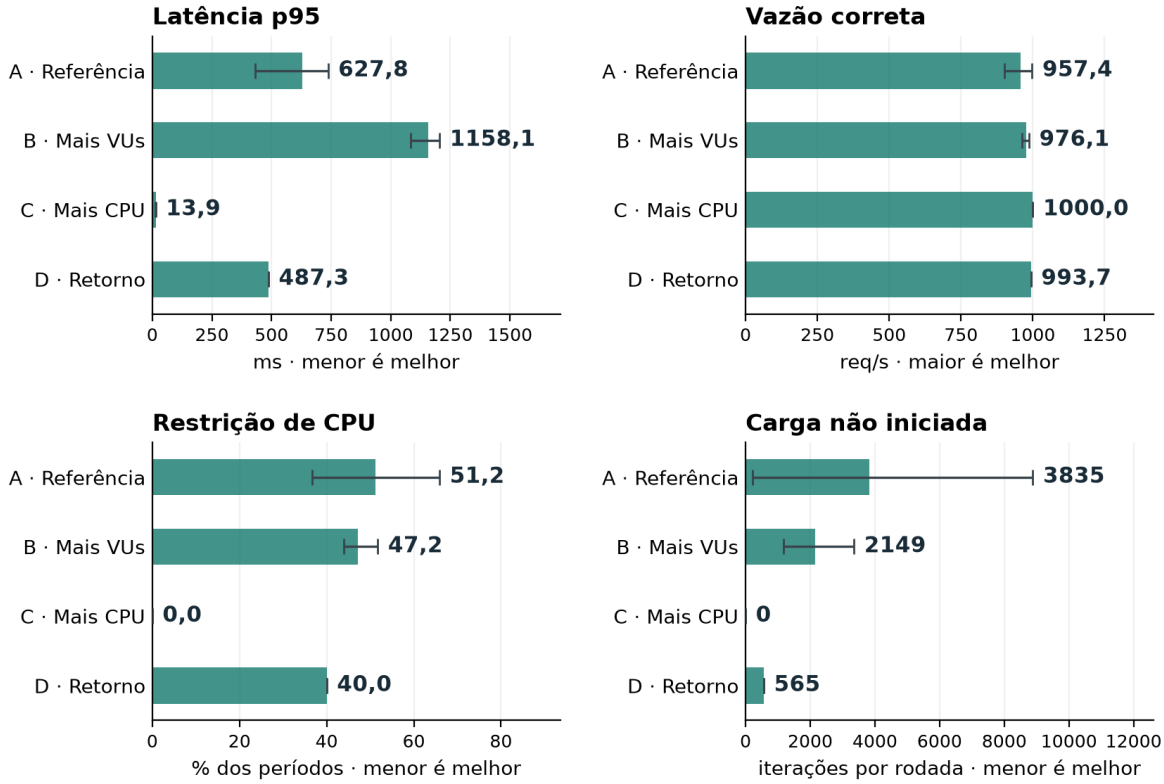
Fonte: elaboração dos autores. Cada célula mostra a mediana em negrito e o mínimo–máximo abaixo (n = 3). Colunas: sessões distintas; não são uma curva temporal controlada. Feign, JVM e plano de energia mudaram entre coletas. WebFlux teve descartes em 20, 29 e 30/10.

Os resultados anteriores são distintos. Em 15/10, o WebFlux teve mediana de p95 de 19,156 ms a 1.000 pedidos/s, sem descartes. Em 20/10, essa mediana passou a 603,287 ms, com 14.923 descartes nas três repetições; em 29/10, a referência apresentou 627,822 ms e 12.927 descartes; em 30/10, 580,591 ms e 6.833 descartes. Os percentis dessas sessões descrevem requisições executadas sob entrega parcial, não uma população idêntica à dos braços que sustentaram toda a chegada.

A coleta de 31/10 demonstra que a implementação reativa conseguiu atender o alvo com a cota de 2 CPUs naquele ambiente. Isso impede concluir, a partir dos dias anteriores, que o framework seria intrinsecamente incapaz de alcançar 1.000 pedidos/s ou exigiria sempre 4 CPUs. Também não se atribui a melhora somente à JVM: 30 e 31/10 já utilizavam a mesma versão, enquanto o plano de energia e o estado do host diferiam. A inicialização antecipada adicionada ao Feign não é uma correção no caminho de execução WebFlux.

4.4 Evidências do diagnóstico de CPU

Figura 4 – Diagnóstico histórico do WebFlux em 29 de outubro



Fonte: elaboração dos autores. Barras e números: medianas; hastes: mínimo–máximo, não intervalo de confiança. A: 2 CPUs/500 VUs; B: 2 CPUs/1.000 VUs; C: 4 CPUs/500 VUs (n = 3 em A–C). D: retorno a 2 CPUs/500 VUs (n = 1). ActiveProcessorCount=2 permanece fixo. Restrição é percentual de períodos, não tempo perdido.

Na referência de 29/10, entre 36,62% e 65,91% dos períodos observados do WebFlux tiveram restrição de CPU. Com cota de 4 CPUs, as três rodadas apresentaram p95 entre 13,631 e 14,167 ms, mediana de 13,874 ms, zero descartes e nenhum incremento do contador de restrição. Na volta final à cota de 2 CPUs, o p95 foi 487,334 ms, com 565 descartes e restrição em 40,02% dos períodos. Essa convergência sustenta sensibilidade à cota naquele estado da bancada, sem explicar isoladamente toda diferença entre dias.

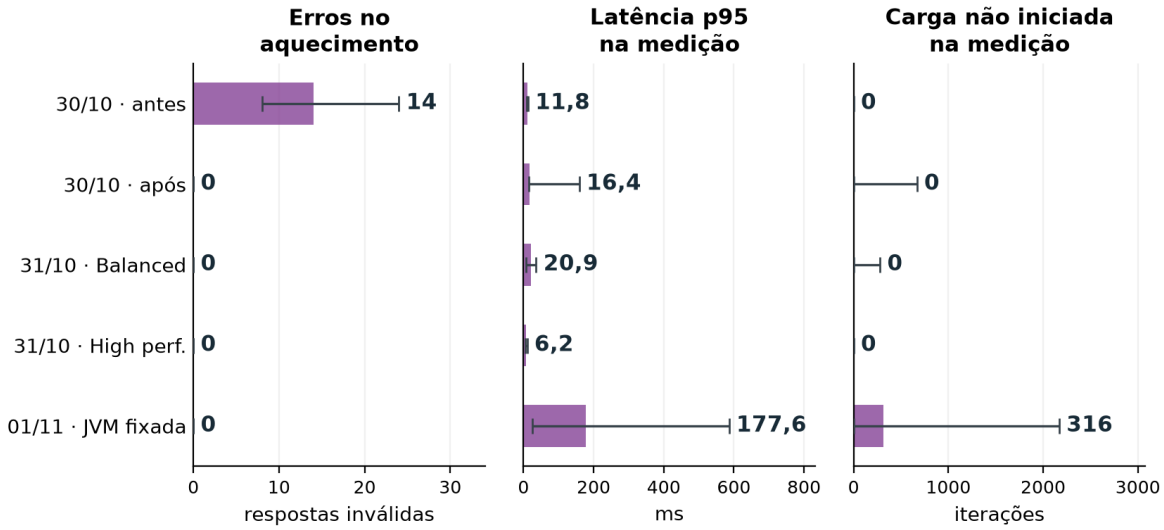
Dobrar somente os VUs para 1.000 manteve 6.673 descartes, uma resposta inválida e mediana de p95 de 1.158,114 ms. Portanto, ampliar o número de operações simultâneas admitidas pelo gerador não eliminou a degradação. A intervenção de 4 CPUs não é uma comparação justa de eficiência com MVC limitado a 2 CPUs, e a sequência não intercalada limita inferências causais. Os perfis instrumentados e a volta final têm apenas uma repetição por condição.

4.5 Validação do Feign e dependências

No aquecimento da matriz de 30/10 ocorreram 186 respostas inválidas, todas em oito das nove rodadas Feign de pedidos: 117 a 200/s, 23 a 500/s e 46 a 1.000/s. As contagens vêm de requests menos successful_requests, não do número de linhas de log. O log preservado registra DecodeException com a mensagem 'messageConverters' must not be empty, mas sua captura é incompleta e não permite associar individualmente todas as falhas a uma causa.

O código de FeignHttpMessageConverters 5.0.1 atribui uma lista vazia ao campo compartilhado antes de preenchê-la, sem sincronização explícita nesse caminho (Spring Cloud, 2026). Isso sustenta a hipótese de uma corrida na primeira inicialização. FeignDecoderWarmup antecipa o preenchimento nos contextos dos clientes; testes de contexto confirmam conversores preenchidos após inicialização e o estado não inicializado sem a intervenção. Esses testes verificam a configuração e o mecanismo de prevenção, sem reproduzir todos os escalonamentos concorrentes possíveis.

Figura 5 – Validação do OpenFeign a 1.000 pedidos por segundo



Fonte: elaboração dos autores. Barras e números: medianas de três repetições; hastes: mínimo–máximo. Antes/após referem-se à inicialização dos conversores. Zero erros não significa ausência de descartes. Planos de energia e JVM diferem entre lotes; nenhuma ligação entre condições é presumida.

Na validação de 30/10 após a mudança, três rodadas Feign a 1.000/s tiveram zero erros em 119.018 requisições de aquecimento. Na matriz completa de 31/10 também não houve erros de aquecimento. Em 01/11, com a JVM histórica, a matriz Feign de nove rodadas registrou zero erros em 198.890 requisições de aquecimento e 456.522 requisições de medição. A convergência é compatível com a eficácia da inicialização antecipada nas condições testadas; não demonstra que todas as falhas antigas, cujas causas não eram registradas, tinham a mesma origem.

Ausência de erro não equivale a estabilidade de latência: em 01/11 os p95 Feign a 1.000/s foram 586,134, 177,587 e 25,122 ms, com 2.169, 316 e zero descartes. Os recursos registrados mostraram alta utilização do serviço de estoque na primeira rodada, compatível com contenção na dependência compartilhada. Como não houve intervenção isolada no estoque nem repetição contemporânea de todos os braços, esse lote não estima um efeito causal da JVM e não é usado para classificar frameworks.

As sessões históricas ainda registram falhas de dependências: houve IllegalReferenceCountException no caminho Netty/R2DBC em 20/10 e na intervenção de 1.000 VUs em 29/10; a última teve uma resposta inválida na medição. Outra rodada do diagnóstico apresentou 335 falhas de aquecimento com timeouts de aquisição R2DBC. As referências lentas de 29/10 não exibiram aquela exceção nos logs consultados. Logo, ela não explica sozinha a degradação, e sua origem interna permanece sem isolamento. A nova coleta sem falhas não comprova correção desse defeito específico.

4.6 Interpretação para adoção das pilhas

Os resultados não contradizem a proposta reativa. As operações avaliadas usam banco e rede locais, consultas curtas e dependências sequenciais. Não foram introduzidas esperas longas e imprevisíveis controladas nem várias chamadas remotas independentes. O modelo reativo pode alterar ocupação de threads sem reduzir serialização, protocolo, validação e persistência. O estudo não demonstra economia geral de memória ou CPU de uma pilha, pois sua conclusão principal se baseia em latência, entrega e correção.

Para decidir sobre um serviço semelhante, é mais informativo verificar atendimento da demanda, dispersão e falhas do que escolher pelo nome do framework. Em 31/10, ambas as pilhas atenderam os alvos; MVC apresentou menores medianas de p95 no recorte. Nos dias de saturação, o conjunto de recursos e dependências teve efeito operacional muito maior do que diferenças submilissegundo de SQL. Essa observação reforça a necessidade de medir a implementação completa no ambiente de interesse.

5 AMEAÇAS À VALIDADE

O host compartilhado, o Docker/WSL2, o gerador local e os bancos concorrentes limitam a validade interna. Planos de energia, condições térmicas e processos do Windows não foram controlados como fatores independentes. A mudança de JVM por tag móvel e a correção do transporte Feign impedem tratar sessões como replicações de uma configuração única. Os

manifestos preservam essas diferenças; digests fixados posteriormente não tornam retrospectivamente todas as imagens equivalentes.

A apresentação prioriza a matriz completa revisada por corresponder à implementação corrigida e ao plano High performance, mas mantém os resultados desfavoráveis anteriores e posteriores. Essa regra é explicitada para reduzir o risco de seleção oportunista. A coleta de 01/11 é parcial, e as intervenções de CPU foram sequenciais. Não há desenho fatorial que separe os efeitos de JVM, energia, transporte e estado do host.

A validade estatística é limitada a três repetições por condição nas matrizes e intervenções principais; perfis e retorno final têm n=1. Milhões de requisições não equivalem a milhões de replicações independentes. Não são usadas significância, equivalência ou intervalos bootstrap para justificar um vencedor. Pontos individuais, medianas e amplitude descrevem o que foi observado, incluindo alertas temporais.

A validade de construção depende do contrato de resposta implementado e do instrumento k6 modificado localmente. A conferência das amostras é interna, não certificação externa. Requisições não iniciadas não integram a distribuição de latência; bloqueio de conexão está fora da métrica principal. A validade externa é restrita por dados sintéticos, base pequena e rede virtual local. Não foram avaliados TLS, autenticação, grandes bases, múltiplas regiões, falhas distribuídas controladas ou MVC com virtual threads. As cotas por container não representam o custo total da solução em nuvem.

6 CONCLUSÃO

Na matriz completa revisada, MVC/JDBC e WebFlux/R2DBC atenderam as taxas SQL e de pedidos sem erros ou descartes. MVC apresentou menores medianas de p95 nas seis condições SQL, com diferenças inferiores a um milissegundo; em pedidos a 1.000/s, a mediana foi 10,414 ms no WebFlux e aproximadamente 6 ms nos dois clientes MVC. Essas diferenças descrevem as pilhas e versões testadas, sem estabelecer superioridade universal.

O histórico acrescenta uma conclusão metodológica: uma comparação confiável precisa registrar também perdas, dependências, transporte efetivo, JVM e recursos. A sensibilidade à cota de CPU observada em 29/10 e o atendimento completo com 2 CPUs em 31/10 mostram que uma sessão isolada não caracteriza um limite intrínseco do WebFlux. A inicialização antecipada dos conversores Feign foi acompanhada de ausência de falhas nas validações posteriores, mas não eliminou toda variabilidade de capacidade. Trabalhos futuros devem

separar o gerador, intercalar intervenções e controlar fatores do ambiente antes de generalizar os resultados para produção.

DISPONIBILIDADE DOS DADOS

O material suplementar contém resultados por rodada, pontos usados nas figuras, scripts de regeneração e conferência, metadados, hashes de fontes e registros de diagnóstico. O arquivo de dados brutos preserva 302 séries de medição: 282 rodadas principais e 20 testes curtos identificados separadamente. Resumos de aquecimento são preservados no suplemento; suas séries brutas não integram esse arquivo. Os lotes históricos são mantidos junto aos sete novos lotes de 30/10 a 01/11. Fontes atuais são distinguidas da cópia histórica, e os manifestos registram a proveniência de cada coleta. Imagens binárias Docker não estão incluídas. O depósito público deve ser identificado como preprint, sem pressupor revisão por pares ou DOI antes da publicação.

REFERÊNCIAS

DEAN, Jeffrey; BARROSO, Luiz André. The tail at scale. Communications of the ACM, v. 56, n. 2, p. 74–80, 2013. Disponível em: https://research.google/pubs/the-tail-at-scale/. Acesso em: 27 out. 2026.

GRAFANA LABS. Dropped iterations. Grafana k6 documentation, [s.d.]-a. Disponível em: https://grafana.com/docs/k6/latest/using-k6/scenarios/concepts/dropped-iterations/. Acesso em: 27 out. 2026.

GRAFANA LABS. Open and closed models. Grafana k6 documentation, [s.d.]-b. Disponível em: https://grafana.com/docs/k6/latest/using-k6/scenarios/concepts/open-vs-closed/. Acesso em: 27 out. 2026.

HALE, Ben et al. R2DBC — Reactive Relational Database Connectivity. Especificação, versão 1.0.0.RELEASE, 25 abr. 2022. Disponível em: https://r2dbc.io/spec/1.0.0.RELEASE/spec/html/. Acesso em: 27 out. 2026.

LINUX KERNEL. Control Group v2. The Linux Kernel documentation, versão 6.6, 2023. Disponível em: https://www.kernel.org/doc/html/v6.6/admin-guide/cgroup-v2.html. Acesso em: 29 out. 2026.

MOCHNIEJ, Kacper; BADUROWICZ, Marcin. Performance comparison of microservices written using reactive and imperative approaches. Journal of Computer Sciences Institute, v. 28, p. 242–247, 2023. DOI: 10.35784/jcsi.3698. Disponível em: https://ph.pollub.pl/index.php/jcsi/article/view/3698. Acesso em: 27 out. 2026.

ORACLE. Package java.sql. Java Platform Standard Edition 21 API Specification, [s.d.]. Disponível em: https://docs.oracle.com/en/java/javase/21/docs/api/java.sql/java/sql/package-summary.html. Acesso em: 27 out. 2026.

REACTIVE STREAMS. Reactive Streams. [S.l.], [s.d.]. Disponível em: https://www.reactive-streams.org/. Acesso em: 27 out. 2026.

SPRING. Overview. Spring Framework Reference Documentation: Spring WebFlux, [s.d.]-a. Disponível em: https://docs.spring.io/spring-framework/reference/web/webflux/new-framework.html. Acesso em: 29 out. 2026.

SPRING. Spring Web MVC. Spring Framework Reference Documentation, [s.d.]-b. Disponível em: https://docs.spring.io/spring-framework/reference/web/webmvc.html. Acesso em: 27 out. 2026.

SPRING. Spring Cloud OpenFeign features. Spring Cloud OpenFeign Reference Documentation, [s.d.]-c. Disponível em: https://docs.spring.io/spring-cloud-openfeign/reference/spring-cloud-openfeign.html. Acesso em: 27 out. 2026.

SPRING CLOUD. FeignHttpMessageConverters.java. Spring Cloud OpenFeign, versão 5.0.1, 2026. Disponível em: https://github.com/spring-cloud/spring-cloud-openfeign/blob/v5.0.1/spring-cloud-openfeign-core/src/main/java/org/springframework/cloud/openfeign/support/FeignHttpMessageConverters.java. Acesso em: 1 nov. 2026.